

AI 100 講 — 補充單元

CLI Agent

AI 接管你的電腦

你打字 → AI 動手

按 → 或空白鍵開始

今天的旅程

1

Demo 震撼

看 AI 兩分鐘從零建網站

2

上手：安裝 + 基本操作

AI 看檔案、改檔案

3

核心概念：CLAUDE.md / Skills / MCP

給 AI 規則、裝技能、接外部工具

4

連上外部世界：GitHub

AI 推 code、部署網站

5

Permission + 安全 + 額度

信任 vs 速度 vs 荷包

從 S01 到 S03：你的角色在變

S01

搬運工

問 AI → 複製 → 手動
貼
你搬 AI 的產出

S02

老闆

描述 → AI 交貨 → 你
驗收
說就好

S03

建造者

你下令 → AI 在你電腦
上蓋房子
從零到上線

從「手動」到「自主」的演化



比喻

S01 = 你是廚師，AI 是食譜書

S02 = 你描述菜色，AI 端上桌

S03 = 你說「我餓了」，AI 進廚房煮完洗完

0:00 – 0:15

Demo 震撼

先看效果，再學怎麼做

Demo 1：一句話建站

A terminal window titled "Terminal - claude" with a dark background. The prompt is "user@mac ~ \$ claude". Below it is a text input field containing "Claude Code v1.0.0 |". At the bottom, a green prompt character ">" is followed by the text "用一句話幫我搭建一個網站，部署到 GitHub Pages" with a cursor at the end.

```
Terminal - claude

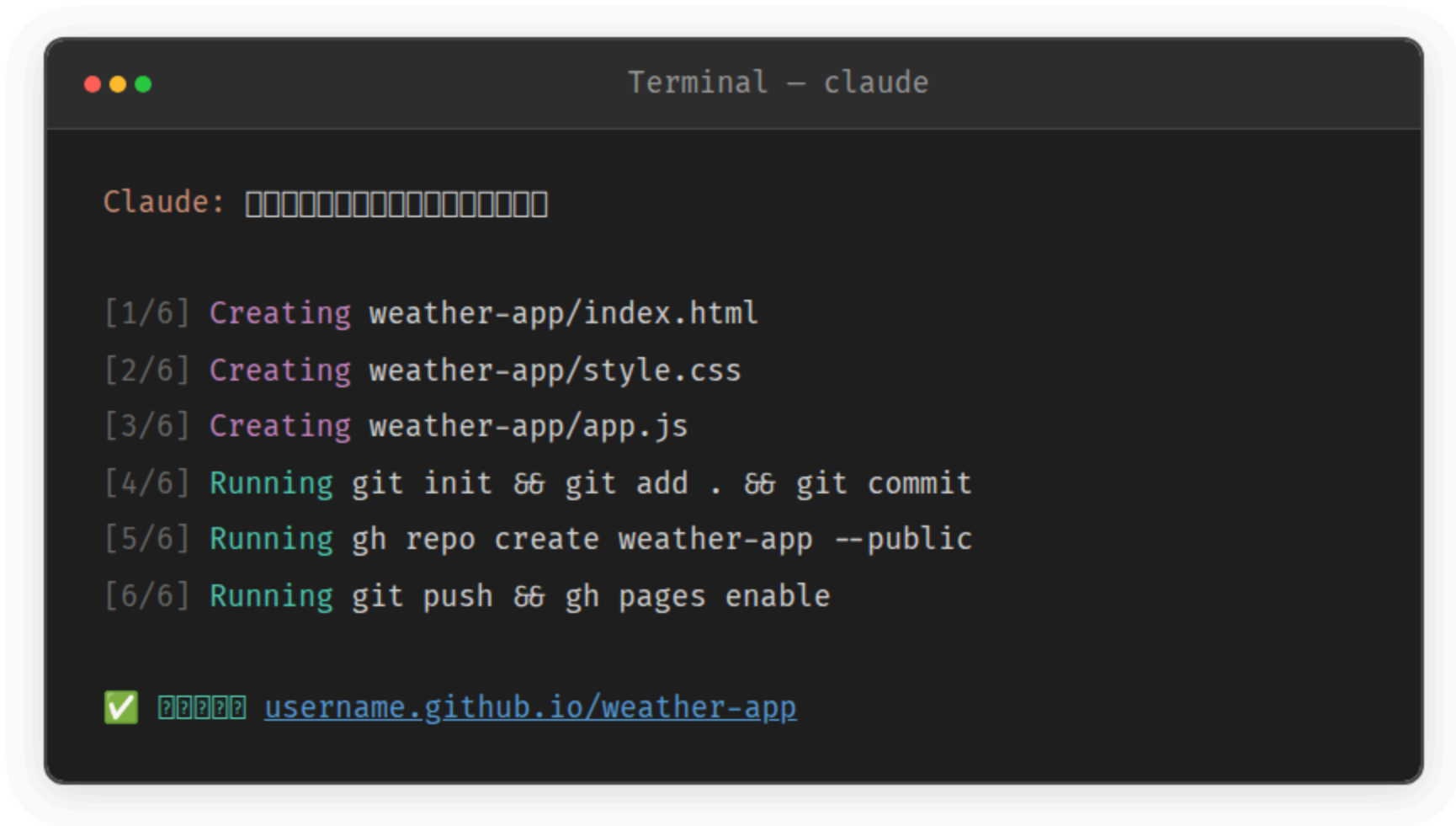
user@mac ~ $ claude

Claude Code v1.0.0 |

> 用一句話幫我搭建一個網站，部署到 GitHub Pages
```

一行中文，不需要任何技術名詞

AI 自己動手了



```
Terminal - claude

Claude: []

[1/6] Creating weather-app/index.html
[2/6] Creating weather-app/style.css
[3/6] Creating weather-app/app.js
[4/6] Running git init && git add . && git commit
[5/6] Running gh repo create weather-app --public
[6/6] Running git push && gh pages enable

✓ [] username.github.io/weather-app
```

打開瀏覽器 — 它真的上線了



從空資料夾到上線網站，2 分鐘

Demo 2：環境設定也行



Terminal – claude

> 我想在 GitHub 上 SSH 怎麼辦

Claude: 我想在 GitHub 上 SSH 怎麼辦

Running `ssh-keygen -t ed25519 -C "you@email.com"`

Running `pbcopy < ~/.ssh/id_ed25519.pub`

我想在 GitHub 上 SSH 怎麼辦 → SSH Keys 頁面

Running `ssh -T git@github.com`

Hi username! You've been authenticated.

剛才發生了什麼？

1. 你用**中文**說了一句話
2. AI 自己**建檔、寫 code、執行指令**
3. 成品直接**上線**，你的瀏覽器能看到

你沒有寫任何一行程式碼

0:15 – 0:45

上手

安裝 + 你的第一次互動

全班一起裝

安裝 Claude Code

S03 補充單元



```
Terminal

user@mac ~ $ npm install -g @anthropic-ai/claude-code

added 1 package in 12s

user@mac ~ $ claude
```

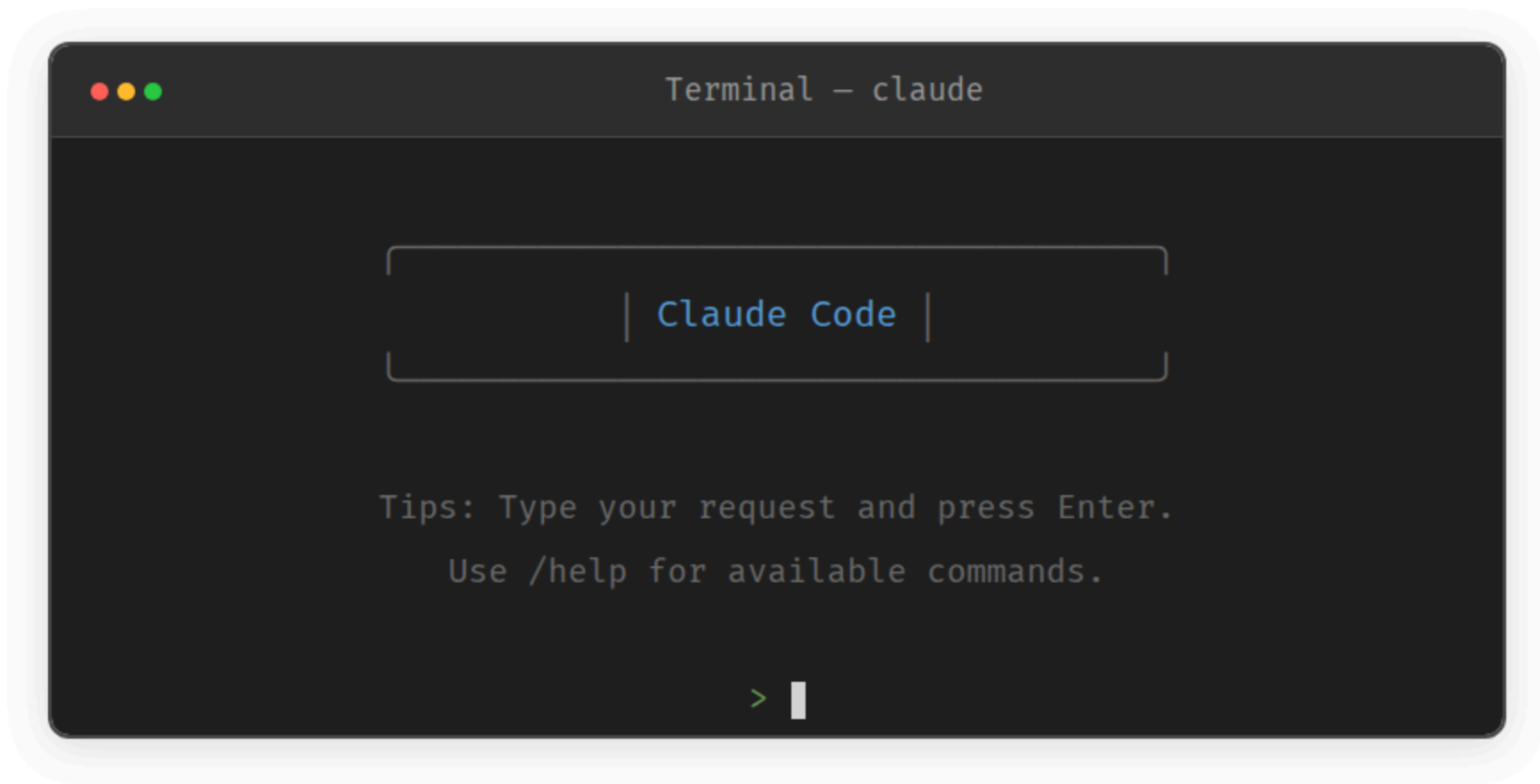
前提

需要 Node.js 18+ 和 Anthropic 帳號 (\$20/月 Pro)

QR Code → 資源頁有安裝步驟教學

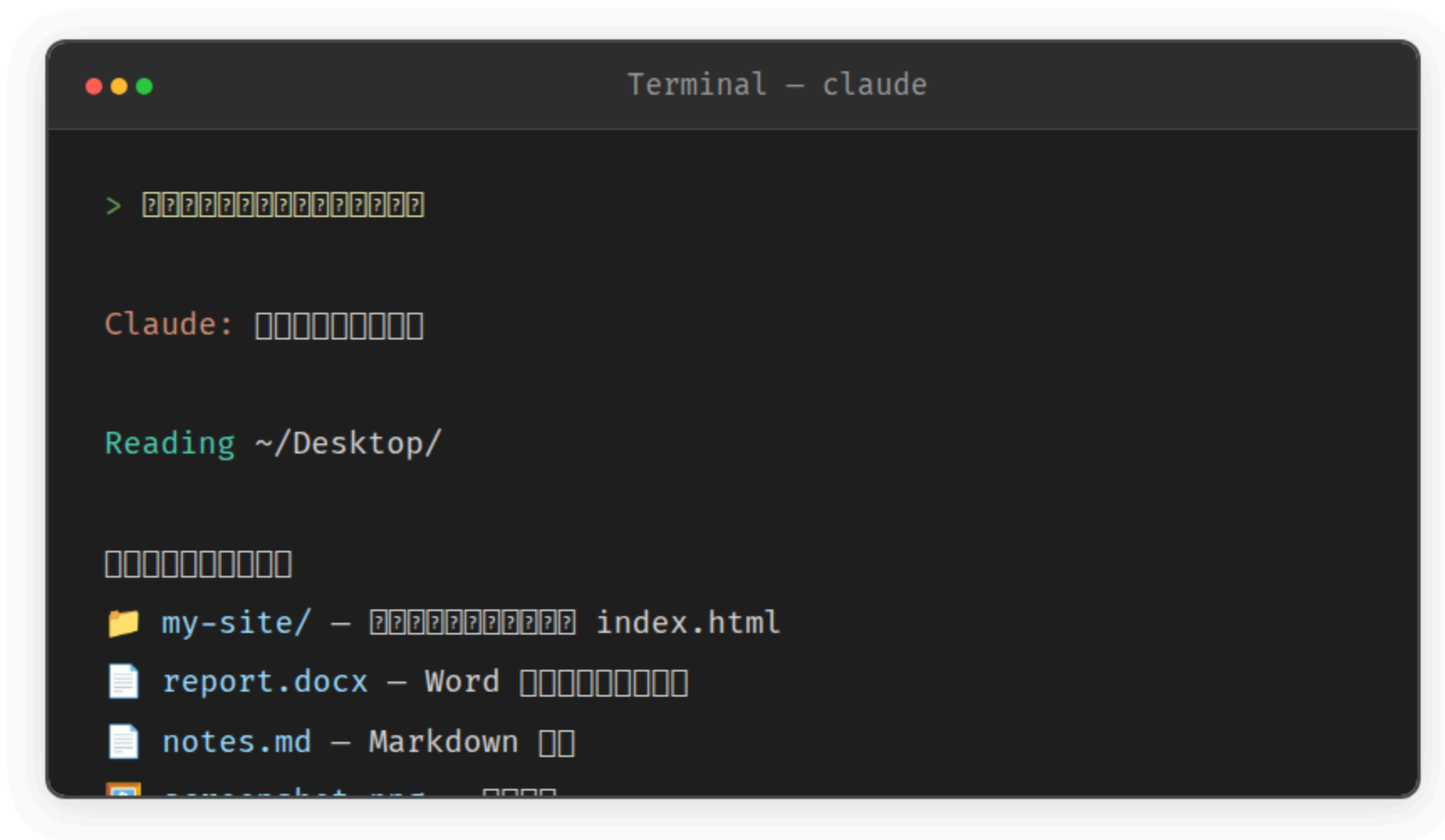
確認安裝成功

看到這個畫面就對了

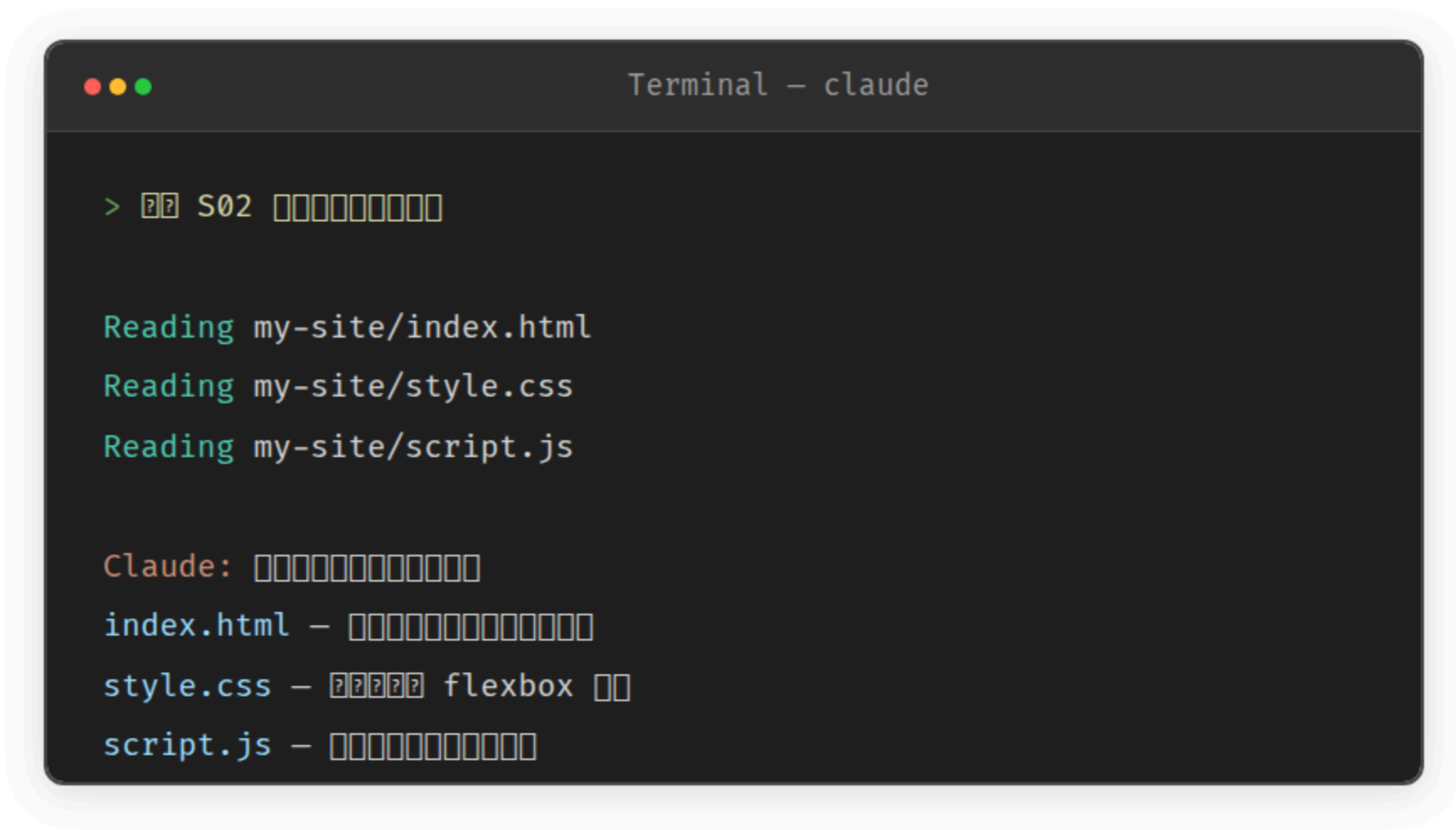




AI 看我的檔案



再試一個



```
Terminal - claude

> ?? S02 [][][][]

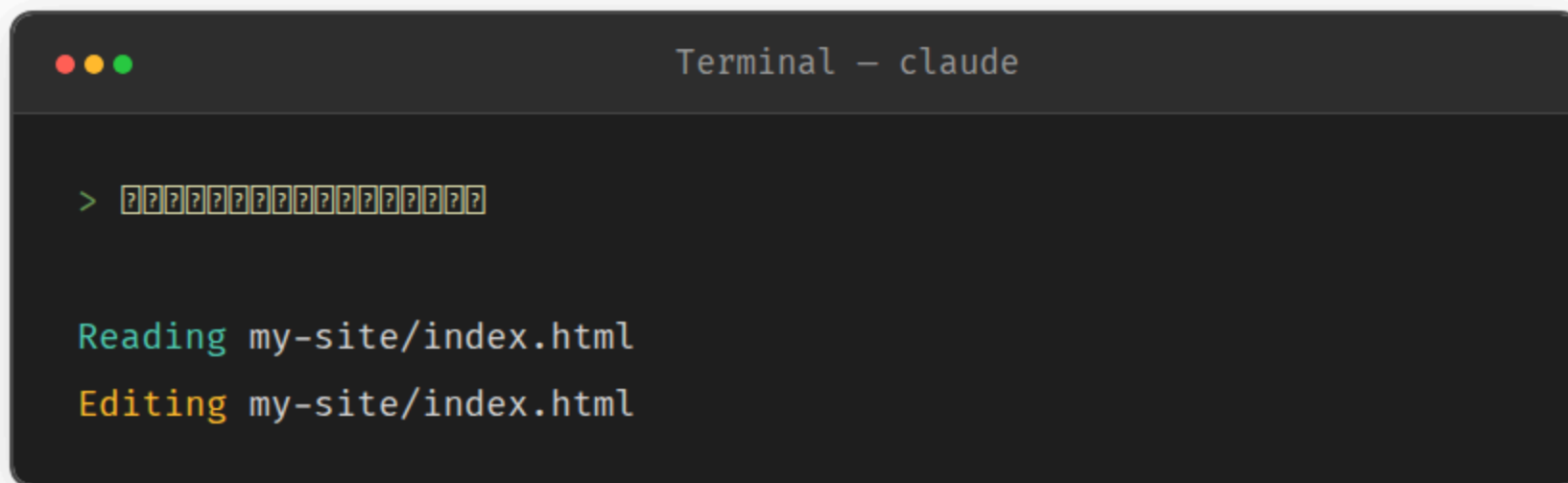
Reading my-site/index.html
Reading my-site/style.css
Reading my-site/script.js

Claude: [][][][]
index.html - [][][][]
style.css - [?] flexbox []
script.js - [][]
```

「它能看到我的東西欸」

★★ 練習

AI 改我的檔案



```
Terminal - claude

> [?]

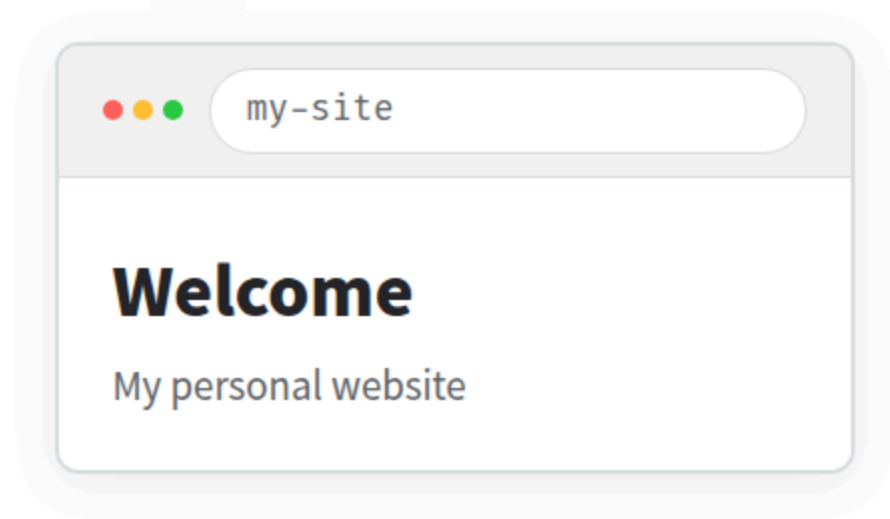
Reading my-site/index.html
Editing my-site/index.html
```

注意

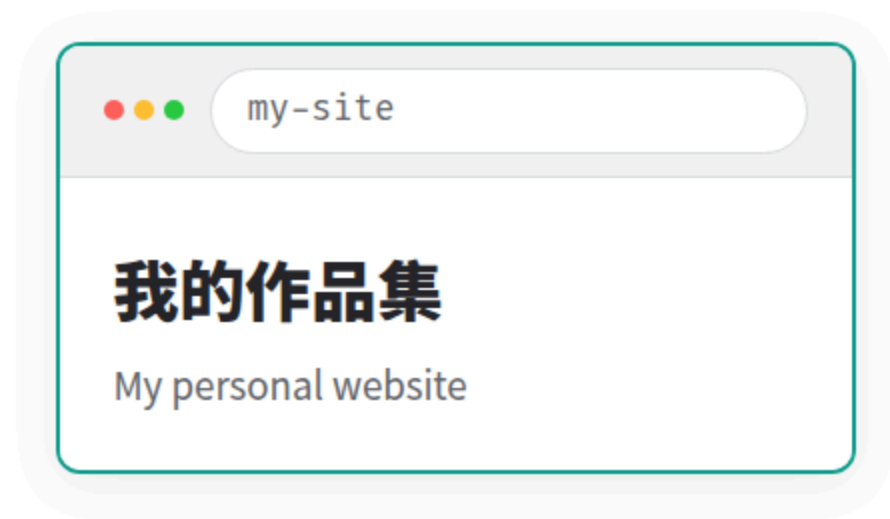
AI 會先顯示要改什麼，等你確認才真的改。就像有人問你：「這樣改可以嗎？」

打開檔案 — 真的改了！

改之前



改之後



再試：「加一個 footer，寫上版權 2026」

「它直接改了我的檔案！」

到目前為止你做了什麼？

★ 讀 — AI 看了你的檔案，解釋給你聽

★★ 改 — AI 修改了你的檔案（經過你同意）

接下來：AI 能不能幫你「做一件事」？



休息 10 分鐘

下半場：核心概念 + 完成任務

0:55 – 1:30

核心概念

CLAUDE.md / Skills / MCP

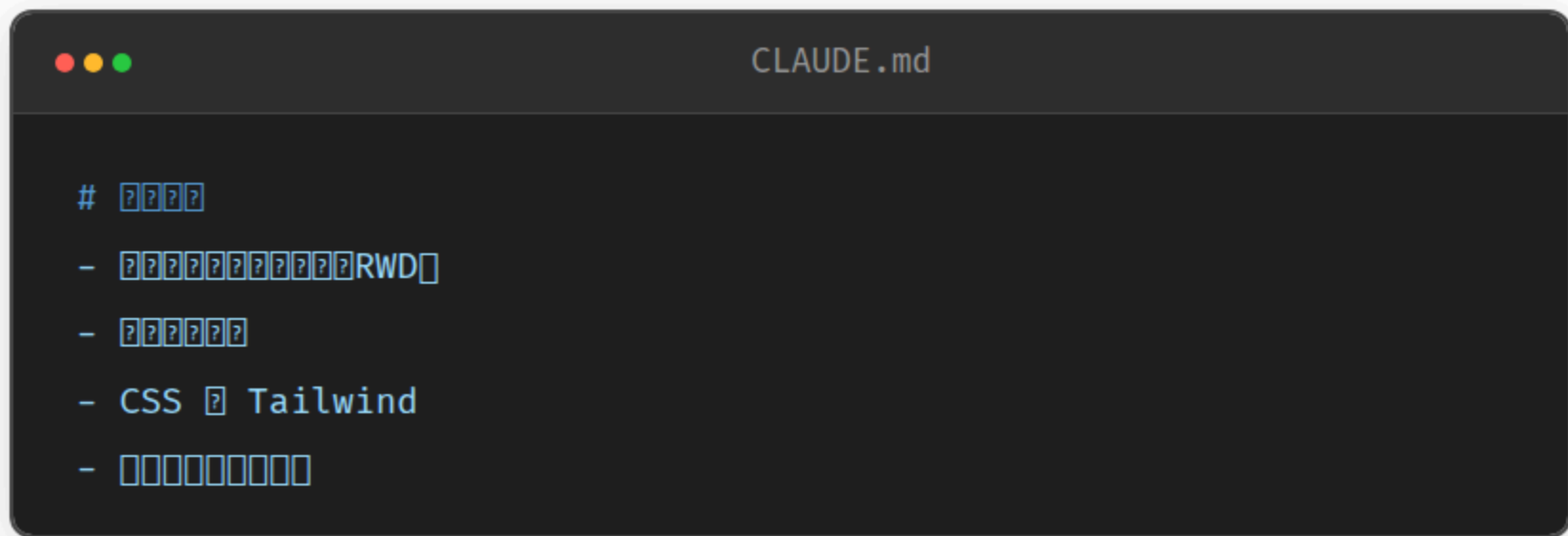
CLAUDE.md — 新員工的 SOP 手冊

比喻

你請了一個超強的助理，但他不知道你的規矩。

CLAUDE.md = 你寫給 AI 的工作守則。

每次開新對話，AI 都會先看這份手冊。



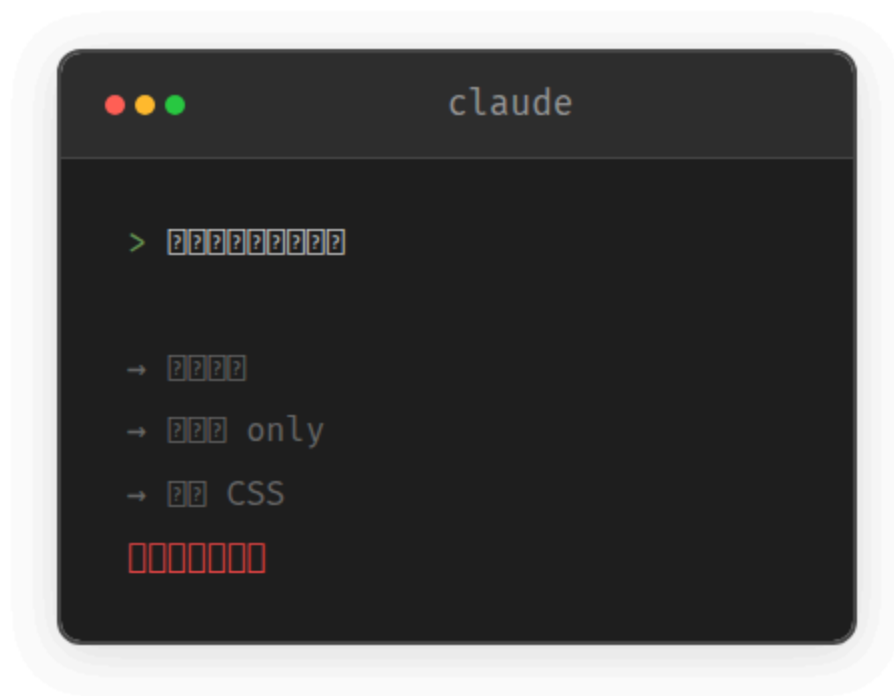
```
CLAUDE.md

# 開發新功能
- 實現後端 API 接口
- 優化數據庫查詢
- CSS 使用 Tailwind
- 部署到雲端
```

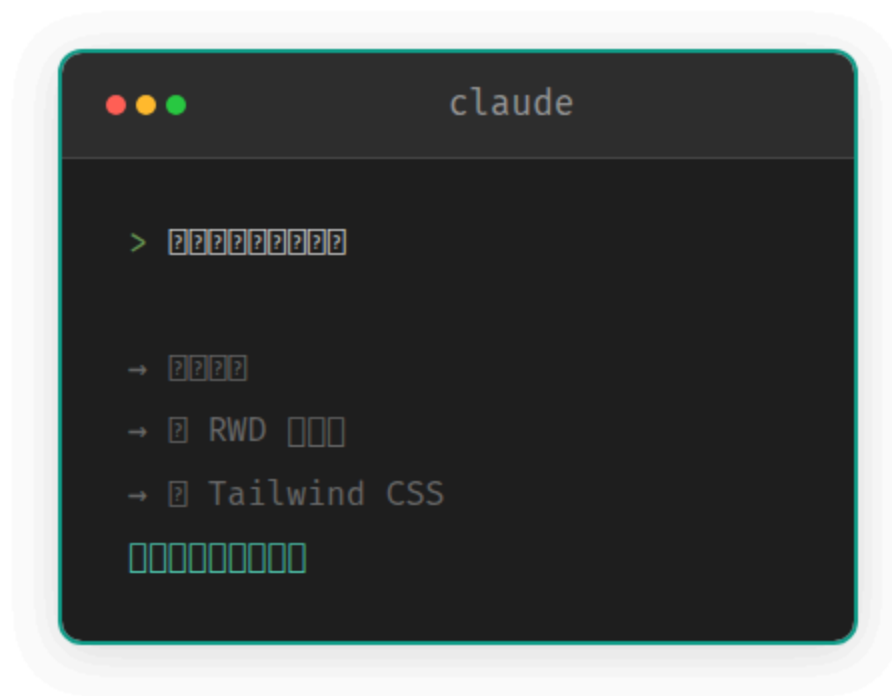
實測

CLAUDE.md 真的有效嗎？

沒有 CLAUDE.md



有 CLAUDE.md



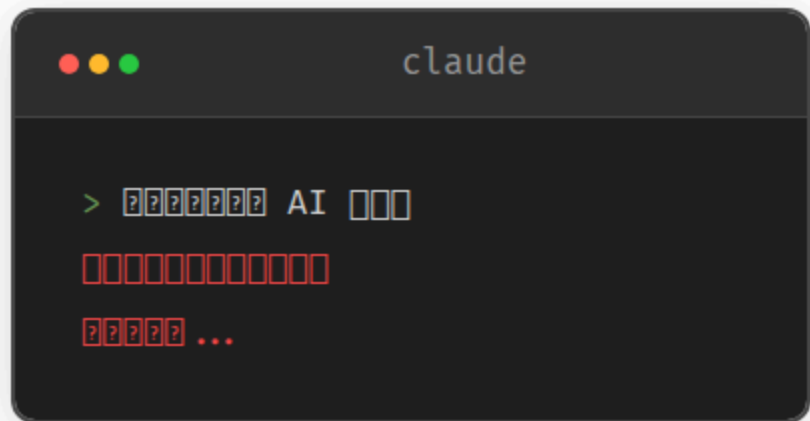
「它記住你的規則了」

Skills — 給 AI 裝 App

比喻

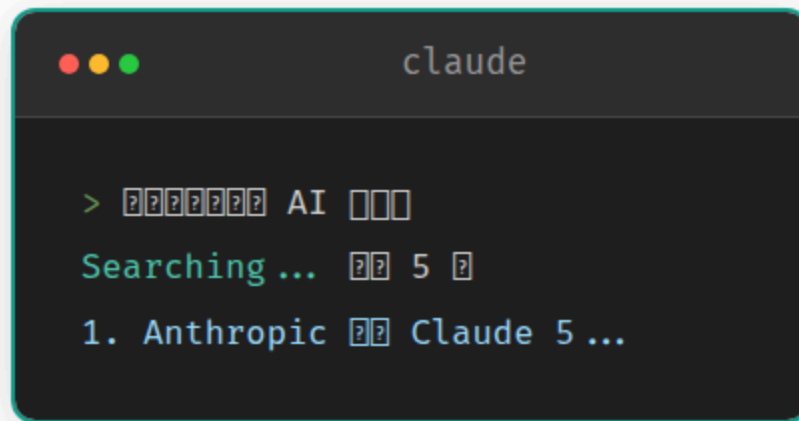
CLAUDE.md 是規則，Skills 是能力。
就像手機裝 App，裝了就多一個功能。

沒有 Skill



```
claude  
  
> 查詢 AI 模型  
[Error: 未找到相關資訊]
```

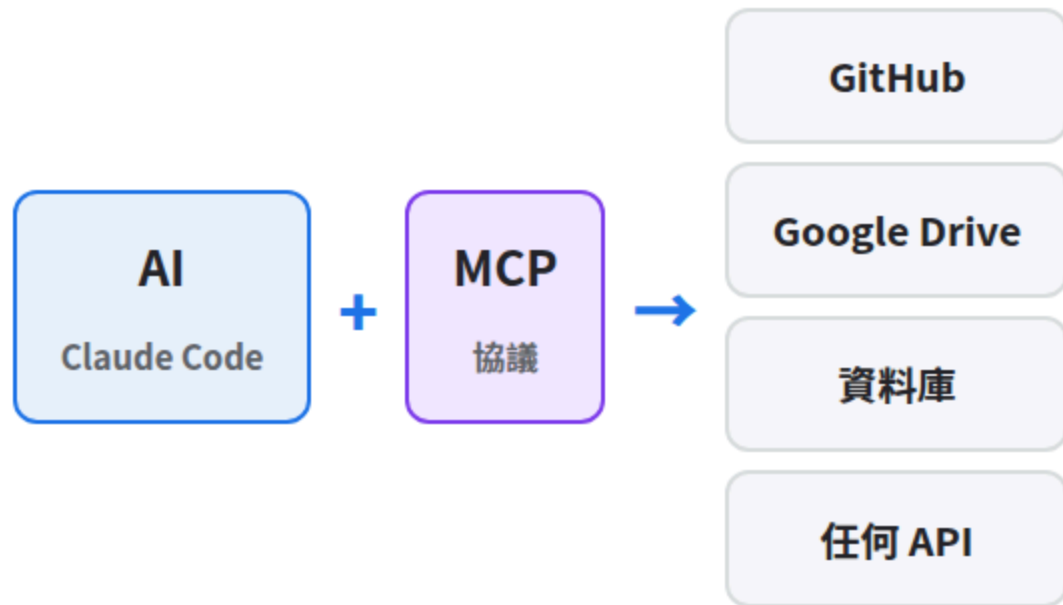
有 web-search Skill



```
claude  
  
> 查詢 AI 模型  
Searching ... 5  
1. Anthropic Claude 5 ...
```

MCP — 萬能插頭

Model Context Protocol — 讓 AI 連接任何外部系統



概念就好

今天不需要設定 MCP，只要知道：

AI 的能力是可以擴充的，不是固定的。

三層架構一次看



CLAUDE.md

規則

AI 的工作守則



Skills

能力

可安裝的功能



MCP

連接

接外部系統

再一次比喻

CLAUDE.md = 公司規章 | Skills = 員工的證照 | MCP = 員工的辦公工具

★★★ 練習

AI 幫我完成一件事

Terminal - claude

```
> [?] [?] [?] [?] [?] [?] Markdown [?] [?] [?] [?] [?] [?]  
[?] [?] PDF
```

```
Claude: [?] [?] [?] [?] [?] [?] [?]
```

```
Reading ~/Desktop/note1.md
```

```
Reading ~/Desktop/note2.md
```

```
Reading ~/Desktop/note3.md
```

```
Creating ~/Desktop/merged-notes.md
```

```
Running npx md-to-pdf merged-notes.md
```

重要觀念：說任務，不說技術

別這樣說

「用 cat 把三個 .md 合併
然後 pipe 到 pandoc
輸出 PDF 格式」

這樣說就好

「把桌面的三份筆記
合併成一份 PDF」

關鍵心法

你說的是**任務**，不是技術指令。

AI 自己決定用什麼工具。你不需要知道 pandoc 是什麼。

「它幫我做到了一件事」

你可以試的任務

別這樣！

A

「把這個資料夾裡的圖片都縮到 800px 寬」

AI 會自己找 ImageMagick 或寫 script

B

「分析這份 CSV，告訴我銷售最好的前 5 名」

AI 會自己用 Python 或 Node.js 分析

C

「幫我把這個 JSON 轉成 Excel」

AI 會自己安裝需要的套件

D

「整理這個資料夾，按日期分類到子資料夾」

AI 會自己寫腳本移動檔案

三大 CLI Agent 比較

現在你體驗過了 Claude Code，來認識其他兩個

Claude Code

一對一助理

最聽話，能設

CLAUDE.md 規則

\$20/月 Pro

適合：深度任務、有規範
的專案

Codex

快速執行者

OpenAI 生態，速度快

\$20/月 Plus

適合：快速 prototype、
GPT 用戶

Antigravity

圖形介面多工

有 GUI，可同時跑多
任務

免費 Preview

適合：新手入門、多工並
行

怎麼選？

情境	建議	原因
剛入門，想免費試	Antigravity	免費 + 圖形介面
有預算，要深度合作	Claude Code	CLAUDE.md + MCP 最強
已有 ChatGPT Plus	Codex	無需額外付費
什麼都想試	都裝	簡單的免費做，複雜的付費做

講師建議

不用糾結。都試試看，選你喜歡的。

它們會做的事 80% 一樣，差別在體驗和生態。



休息 10 分鐘

最後：連上 GitHub + 安全 + 總結

1:40 – 2:20

連上外部世界

AI + GitHub = 無限可能

★★★★ 練習

AI 幫我推上 GitHub

S03 補充單元



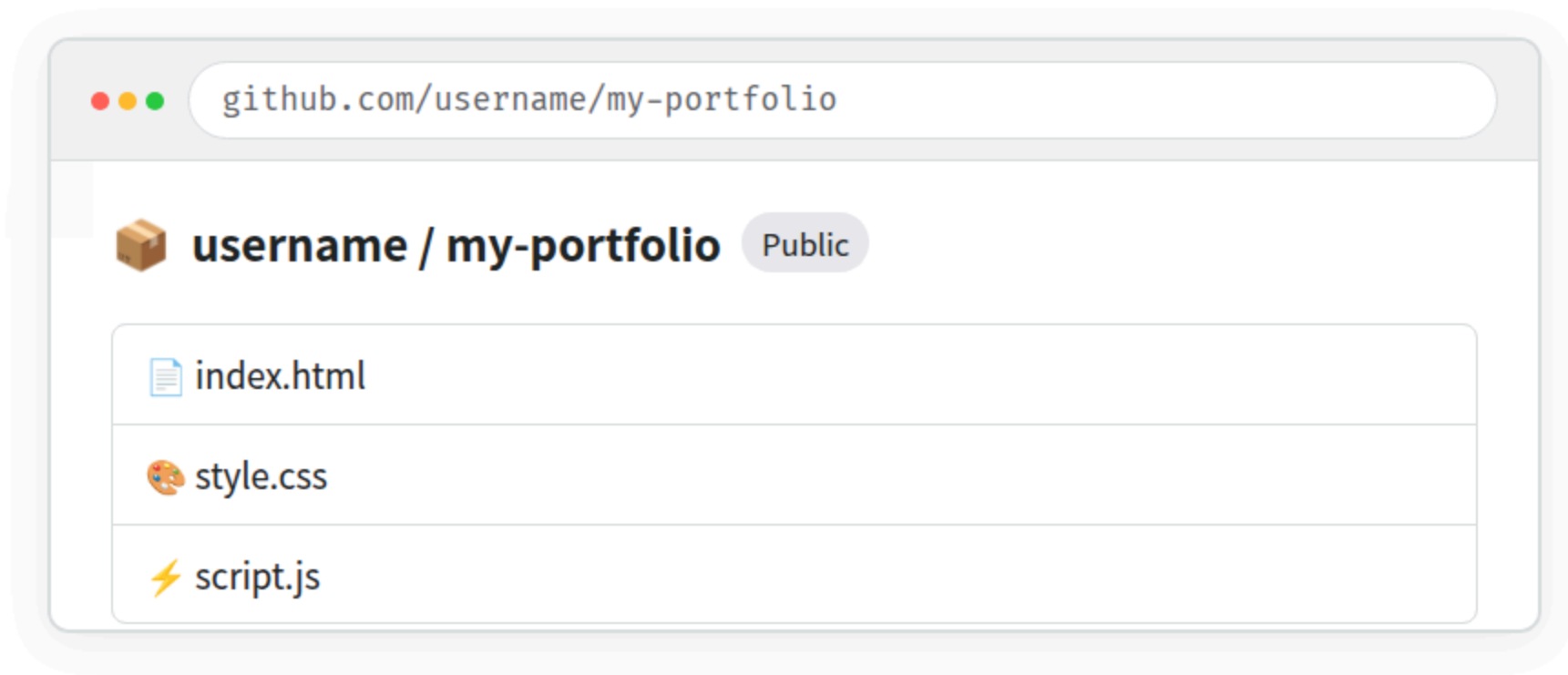
```
Terminal - claude

> ??? GitHub ??? repo ? my-portfolio?
    ??? my-site [][][]

Running cd ~/Desktop/my-site
Running git init
Running git add .
Running git commit -m "Initial commit"
Running gh repo create my-portfolio --public --source=.
Running git push -u origin main

✓ ??? github.com/username/my-portfolio
```

打開 GitHub — 檔案都在！



腦洞大開的瞬間

CLI 能連到的地方，就是 AI 能到的地方。

GitHub、雲端、伺服器、資料庫... 什麼都能推上去。

★★★★★ 大魔王 Demo

從零到上線：一句話搞定



接下來講師操作，你看大螢幕

空資料夾



建立檔案



寫 code



Git push



上線！

AI 正在工作...



Terminal – claude

Claude: 我要建立一個天氣App

[1/8] Creating weather-app/

[2/8] Writing index.html (HTML 模板 + 樣式)

[3/8] Writing style.css (樣式 + 顏色)

[4/8] Writing app.js (API 邏輯)

[5/8] Running git init

[6/8] Running git add . && git commit

[7/8] Running gh repo create --public --push

[8/8] Running gh api repos/.../pages -X POST

✓ 完成

<https://username.github.io/weather-app>

打開瀏覽器 — 它真的在線上



2 分鐘

空資料夾 → 上線的網站

你的感受？

★ 「它能看到我的東西」

★★ 「它直接改了我的檔案」

★★★ 「它幫我做到了一件事」

★★★★ 「它連上 GitHub 了」

★★★★★ 「兩分鐘，從零到上線」

每一步都更「失控」，但你始終是那個按 Yes / No 的人

2:20 – 2:45

Permission + 安全

信任 vs 速度 vs 荷包

Permission 三種模式



安全模式

每個動作都問你

Allow? 一直跳出來

適合：新手、敏感環境



半自動

讀自動、改要確認

READ

WRITE?

適合：日常使用（推薦）



Bypass

AI 不停下來，全速
跑

ALL AUTO

最快，但最燒錢

信任光譜

安全

平衡

全速

每步確認

讀自動 / 寫確認

全自動

信任 ↓ 速度 ↓ 花費 ↓

信任 ↑ 速度 ↑ 花費 ↑

訣竅

剛開始用**半自動**。等你熟悉了，再決定要不要開 Bypass。

Bypass 的代價

正常模式

AI 做一步 → 等你確認
大部分時間在等你按 Y
\$20/月用很久

Bypass 模式

AI 連續做不停
每一步都在燒 token
\$20 可能一個下午就燒完

原則

確定要做什麼才開 Bypass，做完馬上關。
就像計程車跳表 — 你不會開著計程車去逛街。

Bypass 怎麼設才安全？

講師的實際設定：幾乎全開，但「刪除」仍需確認

```
"permissions": {  
  "defaultMode": "bypassPermissions", ← ☐☐☐☐  
  "ask": [  
    "Bash(rm -rf:*)", ← ☐☐☐☐☐☐☐☐  
    "Bash(sudo rm:*)" ← sudo ☐☐☐☐  
  ]  
}
```

⚠ 任務中斷風險

bypass 中斷網 / 關機 / 額度用完 → AI 停在半路，不會自動回滾

檔案可能改到一半、Git 可能 commit 到一半

所以：開 **bypass** 前先寫好 **CLAUDE.md**，不然 AI 沒規則會亂跑

要花多少錢？

方案	費用	適合
Claude Pro	\$20/月	不開 bypass 日常很夠用
Antigravity	免費 (Preview)	入門首選
混合使用	\$0-20	簡單免費做，複雜付費做

💡 查自己用了多少

Claude → claude.ai/settings → Usage

Groq → console.groq.com → Usage

OpenRouter → openrouter.ai/activity

實際用量因人而異，建議自己追蹤一週再評估

安全提醒



1. 刪除操作 — 先看清楚範圍
2. 密碼 / API Key — 不要讓 AI 寫在公開檔案裡
3. 不確定就按 n — AI 不會生氣，它會換方式

什麼時候該按 **n** ?



AI 要刪大量檔案

「rm -rf」或大範圍刪除 → 先問它具體刪什麼



AI 要安裝不認識的套件

看不懂的 npm/pip 套件 → 先問它為什麼需要



AI 要修改系統設定

碰 .zshrc、.bashrc 等 → 先確認改什麼



AI 要推到公開 repo

確認沒有密碼、個資在裡面

2:45 – 3:00

Recap + 預告

今天你學會了

AI 能讀你的檔案 — 看懂你的專案結構

AI 能改你的檔案 — 經過你同意才動手

AI 能幫你完成任務 — 它自己決定用什麼工具

AI 能連上 GitHub — 推 code、建 repo、部署

AI 能從零到上線 — 兩分鐘，空資料夾到網站

帶走的 CLAUDE.md 模板



```
CLAUDE.md

# 專案名稱

## 目標
- 實現一個基於 React 的 Web 應用
- 使用 Tailwind CSS 進行樣式設計

## 技術棧
- 使用 React 作為前端框架
- 使用 Tailwind CSS 進行樣式設計

## 其他
- API Key 需要配置
```

三工具速查表

	Claude Code	Codex	Antigravity
安裝	npm install	npm install	下載 App
介面	Terminal	Terminal	GUI + Terminal
規則檔	CLAUDE.md	AGENTS.md	Settings UI
擴充	Skills + MCP	Tools	Extensions
費用	\$20/月	\$20/月	免費 Preview
強項	深度合作	速度快	視覺化

Claude Code 常用指令



```
# [?]
$ claude # [?]
$ claude "[?]" # [?]

# [?]
> /help # [?]
> /clear # [?]
> /cost # [?]
> /permissions # [?]
```

下一堂預告

S03 你學了

AI 從零建造

S04 你會學

**AI 接手別人的
改成你的**



S03 補充單元

完成！

讀 → 改 → 做 → 推 → 上線

你是建造者，AI 是你的施工隊

ai100.cooperation.tw